SARJ AI · FULL STACK ENGINEER TAKE-HOME

sarjy

A voice concierge for Magic Experience, a tour operator in Dubai. Built by Ahmed Eltawil.

sarjy.magicexperience.ae

Chrome or Safari, on a laptop or a phone. No login and nothing to install: allow the microphone, hold it while you speak and let go to send (or switch to tapping in the settings).

A walkthrough and a teaser narrated by Sarjy, the code book and every decision: github.com/ahmedel-tawil/sarjy

What it does

A traveller talks; Sarjy answers out loud from the operator's live catalogue, checks the weather, and remembers them next time. The four scenarios from the plan all pass on the deployed URL in Chrome and Safari on a Mac and Safari on an iPhone:

1. **Find an activity.** "I'm in Abu Dhabi next week with two kids. What can we do under 400 dirhams?" Two or three real tours with their real prices and links to their pages.
2. **Remember across visits.** "My favourite colour is green, and I don't like heights." Close the tab, come back: "What's my favourite colour?" "Green." Later suggestions leave out the helicopter.
3. **Weather-aware advice.** "Is tomorrow afternoon good for a desert safari?" The real forecast, and a cooler time when it's too hot.
4. **Honest when data is missing.** "How much is the buggy dune bashing tour?" Price on request, never an invented number.

Also: **live availability** ("Is the helicopter flight free next Saturday for two adults?", with the price for the party), a choice of voices, earlier visits whose answers can be played again, and a line that says what Sarjy is doing ("Checking tomorrow's weather in Dubai").

Why these APIs

Sarjy calls **SayTech's assistant API** for tours, prices, policies and availability, and **Open-Meteo** for the weather. Magic Experience is a real operator, run by my brother, and its website runs on SayTech, my own multi-tenant platform, so Sarjy speaks from the catalogue customers book from, with honest gaps ("price on request", "can't check live availability"). A tour concierge in the UAE lives or dies on the heat, so the forecast turns "what can we do?" into "go at six, it's cooler"; Open-Meteo needs no key and covers sixteen days.

The deep dive: latency

From your last word to Sarjy's first

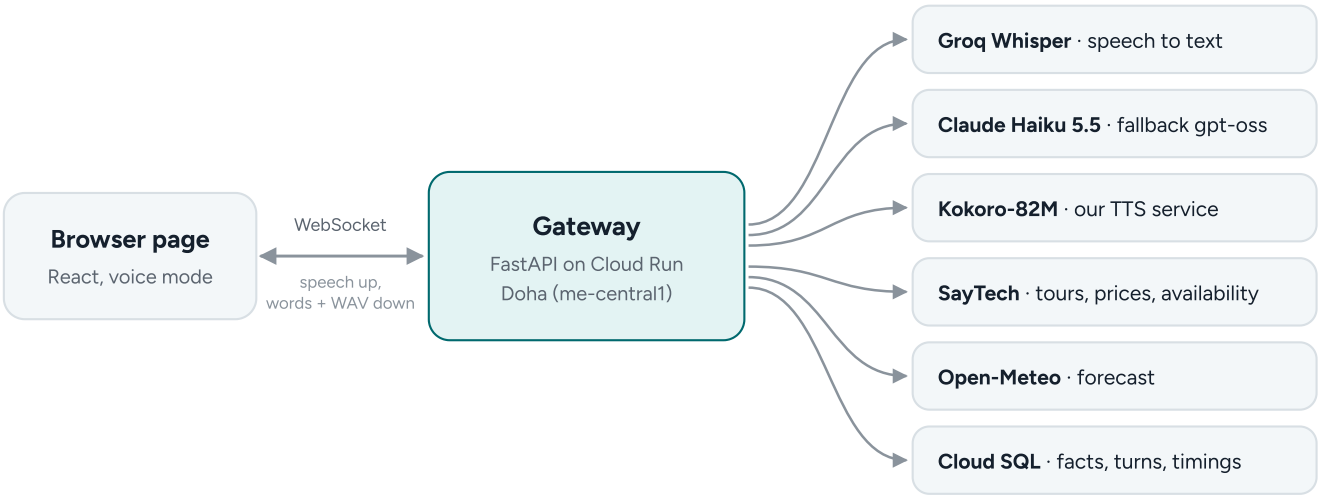
Time to first audio, median of 30 turns: the same ten spoken questions on the deployed service



Measured on every turn with seven marks across the browser and the gateway, shown live in the page, and compared over the same ten spoken questions in nine experiments.

ARCHITECTURE

How it works



One turn, from the moment the traveller stops speaking

1. The page has streamed the question's audio while it was spoken; the end of the question sends `turn_end`.
2. The gateway sends the audio to Whisper and, meanwhile, builds the prompt: the rules and the catalogue's context (cached by Claude), then the traveller's saved facts and the time.
3. Claude streams its answer. When it calls a tool (search, tour details, availability, weather, remember or forget a fact) the gateway runs it, tells the page what it is doing, and asks again with the result.
4. Each sentence goes to TTS the moment it is complete; its WAV goes to the page, which plays the clips back to back on the audio clock and lights up each word as it is spoken.
5. The gateway stores the turn with its timing marks; the page adds its own and shows the waterfall.

Stack

Part	Choice	Why
Speech to text	Groq `whisper-large-v3-turbo`	Fast hosted Whisper, under a second for a question
Model	Claude Haiku 5.5, Groq gpt-oss-120b as fallback	The only model that called its tools and saved facts reliably (experiment 4)
Voice	Kokoro-82M on ONNX Runtime, our own text front end	Full control of latency, sentence by sentence; a fraction of per-character prices
Backend	Python 3.14, FastAPI, Pydantic v2, psycopg 3 with raw SQL	Small, typed, every external service behind one interface with a fake for tests
Frontend	React, Vite, TypeScript, Tailwind, shadcn/ui	Built into the gateway's image: one origin, one URL
Infrastructure	Cloud Run, Cloud SQL, Terraform, keyless CI, a global load balancer for the domain	Deploys on every merge; no long-lived keys; Sarj's code standards enforced from the first commit

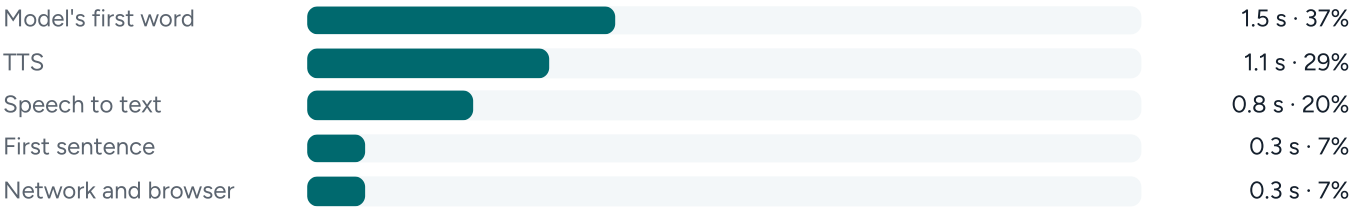
THE DEEP DIVE

Latency: where the time goes and what moved it

Time to first audio runs from the end of the question to the first sound of the answer. Every turn records seven marks on two clocks. Each experiment changed one thing and replayed the same ten spoken questions; three runs on the deployed service (30 turns each, p50 / p95):

Gap	Baseline, 9 Oct	Sentence streaming, 9 Oct	Final, 10 Oct
Speech to text	0.73 / 0.90 s	0.74 / 0.91 s	0.77 / 0.98 s
Model's first word	2.28 / 3.57 s	1.01 / 2.79 s	1.04 / 2.55 s
First sentence complete	0.38 / 0.53 s	0.22 / 0.66 s	0.21 / 0.70 s
TTS to first byte	3.48 / 5.88 s	0.81 / 2.66 s	1.20 / 2.84 s
Network and browser	0.50 / 0.97 s	0.23 / 0.63 s	0.24 / 0.56 s
Time to first audio	**7.89 / 10.67 s**	**3.37 / 6.55 s**	**4.10 / 6.41 s**

Where the time goes now (final run, mean 4.0 s)



The final median sits above the streaming run's because the model opened with longer sentences that day (28% more audio in the first clip), so TTS now tracks the first sentence's length. Turns that opened with a cached sentence answered in 2.3 to 3.2 s.

The experiments

#	Change	Result	Kept
1	Baseline: whole reply synthesised first	7.9 s median; TTS half the wait	—
2	Speak each sentence as soon as it is written	3.4 s median; TTS 3.5 → 0.8 s	**yes**
3	TTS cache in the gateway	1 sentence in 10 a hit; a cached first sentence saves ~1.3 s	**yes**
4	Model choice, with correctness checks	Haiku 18 of 20; the fastest model skipped its tools	**Haiku**
5	Lean vs raw tool payloads	No latency difference	**lean**
6	Warm vs cold instances	A cold TTS added 6 s; waking it as a visit opens hides it	**yes**
7	Region	Doha costs 0.3 to 0.45 s a turn in provider hops vs US East	Doha
8	Kokoro on GPU	Not run: no GPUs in the region	later
9	Prompt caching	No latency change; input cost down two thirds	**for cost**

LESSONS AND NEXT STEPS

What worked

- **Sentence streaming** halved the wait: TTS stopped waiting for the whole reply.
- **A bigger, awake TTS:** 8 vCPU cut a sentence from 3.7 to 2.1 s, and waking it when a visit opens hides a 6 s cold start.
- **Measuring every turn:** stored marks, a replayable script and a live waterfall, so every claim has a run behind it.

What didn't

- **Prompt caching, region, payload size:** little or no latency change, each with the numbers; caching stayed for cost.
- **The TTS cache** helps only turns that open with a sentence it has heard; the medians held.
- **The fastest model** was fast because it skipped its tools; correctness decided.

With another week

1. **A short first sentence:** TTS now scales with it; my estimate is 0.3 to 0.6 s off the median.
2. **A cached "let me check" while tools run,** so a tool turn sounds as quick as a plain one.
3. **Streaming speech to text,** transcribing while the traveller still speaks.
4. **A US region with a GPU for TTS,** measured first from a vantage point in the Gulf.
5. **Hands-free turns and barge-in,** and deploys that hand visits over instead of dropping them.

Decisions and trade-offs

- **Grounded or silent.** Prices and availability come only from SayTech; "price on request" and "unknown availability" are said as such, and each turn keeps its tool results in the history so later turns trust earlier answers.
- **Memory through tools.** The model saves facts with `remember_fact`; they go into every prompt and a "What Sarjy remembers" panel, and "Forget me" deletes them.
- **Our own voice.** Self-hosted Kokoro instead of a per-character API: sentence-level control and cost, at the price of running it.
- **One region, Doha.** Close to Gulf visitors; a US region would save a few hundred milliseconds of provider hops, not worth a migration a day before the deadline.
- **Known limit:** a deploy drops the visits in progress, so merges wait for quiet times.

Cost per conversation

About **$0.027 (10 fils)** for ten questions: speech to text $0.001, Claude $0.004, our TTS $0.011, the gateway $0.011 at most. Our TTS costs a ninth of ElevenLabs Flash per conversation; keeping it warm ($3.63 a day) pays off from about 44 conversations a day.

How it was built

Sarj's public code and repository standards from the first commit, enforced in hooks and CI. Built with Claude Code under written rules: one task at a time, a plan approved before code, tests and the full check before every commit, nothing deployed without my go, and about a hundred decisions recorded with their reasons.